Universe Reinsurance Platform — Repository Migration & Database Alignment Guide

Switching the deployment source from `Darchville-Analytics/modelling_tool` to `iwadaya/Saudi_re`, seeding the database, and proving the application is linked to — and aligned with — its database.

Version 1.0 — September 2026 — Confidential, for internal IT use only.
Companion to the *Work Environment Deployment Specification v2.0* (July 2026) and the *Redeployment Quick Reference* (August 2026). Where this guide and the v2.0 specification differ, this guide is current.
Source repository: `https://github.com/iwadaya/Saudi_re` (branch `main`). Living companions in the repository: `DEPLOYMENT.md`, `deploy/README.md`, `docs/seed-test-treaties.md`.

---

1. What this guide covers, and what changed

The server was stood up with the v2.0 specification: Ubuntu, the application checked out at `/opt/universe` as the `universe` user, one `.env` file at `/opt/universe/.env`, PM2 running `ecosystem.config.cjs`, nginx terminating TLS in front of port 4000, and PostgreSQL holding the `universe` database. **None of that changes.** What changes is *where the code comes from*: the deployment source moves from the organisation repository `Darchville-Analytics/modelling_tool` to `iwadaya/Saudi_re`.

The new repository contains the full history of the old one plus a maintained deployment kit, so switching is a fast-forward of the same code line, not a migration to a different product. Two things in the new repository behave differently from what the v2.0 specification describes, and both matter on the day you switch:

Area	v2.0 specification (`modelling_tool`)	Now (`iwadaya/Saudi_re`)
Repository and access	Org-owned private repo. Fine-grained PAT with resource owner *Darchville-Analytics*, or a deploy key on `modelling_tool`.	Repo under the personal account **iwadaya**, currently **public**: anonymous HTTPS `fetch` works, no token or key needed. If it is made private again, the old token or deploy key **does not** open it — you need a new fine-grained PAT (resource owner *iwadaya*, repository *Saudi_re*, Contents: read) or a new deploy key added to `Saudi_re` (§3).

Area	v2.0 specification (`modelling_tool`)	Now (`iwadaya/Saudi_re`)
`deploy.sh`	Pulled `main`, ran `npm ci`, built, migrated, reloaded PM2. Run as the app user.	The root `deploy.sh` is now a one-line wrapper that runs **`deploy/deploy.sh`**: fetch/checkout → `npm ci` → build → `migrate:status` + `migrate:up` → **reference-data seed** → `systemctl restart universe` → smoke test. It is written for **sudo** (it drops to `universe` for the Git/npm/build/migrate/seed steps itself), and it reads **`/etc/universe/universe.env`** by default. On a PM2 server you therefore run it with `ENV_FILE=/opt/universe/.env` and `--no-restart`, then reload PM2 yourself (§4, step 5).
Reference-data seeding	Happened on application boot only (`ensureReferenceData`), plus the optional `SEED_ON_DEPLOY=reference` hook.	Still happens on boot, **and** is an explicit deploy step: `node server/src/db/seeds/run.js` (`npm run seed:reference`). It adds the GCC market extras (Saudi/GCC cedants and reinsurers) and refuses to touch contracts, quotes, claims or users. `npm run seed:reference:check` reports the counts without writing anything (§5).
Migrations	Same runner and commands.	Same chain (`_migrations` table keyed by filename). This snapshot ships **149** migration files, the last being `157_quote_offer_status_checks.sql`. Nothing is re-applied; only files your database has not seen run.
Deployment kit	—	New `deploy/` directory: `install-server.sh`, `provision-db.sh`, `deploy.sh`, `smoke-test.sh`, a hardened systemd unit, an nginx site and an environment template — the maintained path for a **fresh** server (§6).
Smoke test	Manual `curl`.	`deploy/smoke-test.sh`: shallow health, deep health (DB round-trip), a non-empty login user list, SPA served, optional real login. Exit 0 = green.

Everything else in the v2.0 specification — sizing, ports, nginx, TLS, backups, secrets, PM2, the security baseline — still applies as written.

> **Two paths.** §4 switches the **existing** server in place (recommended: no re-provisioning, no data movement, ~15 minutes). §6 builds a **fresh** server from the new repository with the deployment kit and moves the data across. Both end at the same checklist in §7.

2. Before you start

Collect these facts first; several steps depend on them.

Item	How to find it	Write it here
Runtime in use	`pm2 status` (as the `universe` user) shows an `online` `universe` process → **PM2** (v2.0). `systemctl status universe` shows `active (running)` → **systemd** (deployment kit).	
Application root and env file	v2.0: `/opt/universe` and `/opt/universe/.env`. Kit: `/opt/universe` and `/etc/universe/universe.env` (symlinked to `/opt/universe/.env`).	

Item	How to find it	Write it here
Currently deployed commit	`sudo -u universe -H git -C /opt/universe log -1 --format='%h %ci %s'` (Git refuses to open a checkout owned by another user — `dubious ownership` — so always run Git as `universe`)	
Current remote	`sudo -u universe -H git -C /opt/universe remote -v` — expect `Darchville-Analytics/modelling_tool`	
Migration state before the switch	`sudo -u universe -H npm run migrate:status --prefix server` from `/opt/universe` → the last lines must read `Pending (0):`. If anything is pending *before* you start, resolve that first.	
Node.js seen by root and by `universe`	`sudo node -v` and `sudo -u universe -H node -v` → both Node 20 or newer, ideally the same version. `deploy/deploy.sh` builds and migrates with the `node` on **root's** PATH (under `sudo`); PM2 runs the app with the **`universe`** user's. If Node was installed with nvm for `universe` only, `sudo node -v` fails and Option A1 dies with `node is not installed` — either install Node system-wide (NodeSource) or use Option A2.	
`pm2` on the `universe` user's PATH	`sudo -u universe -H pm2 -v` → prints a version (v2.0 installed PM2 globally). If it prints `command not found`, use `/opt/universe/node_modules/.bin/pm2` wherever this guide says `pm2`.	
Database	`DATABASE_URL` in the env file (host, port, database name, role). `psql "$DATABASE_URL" -c 'select 1'` must work from the app server.	
Repository visibility	`git ls-remote --heads https://github.com/iwadaya/Saudi_re.git main` from the server without any credentials: prints a commit id → public, §3 can be skipped; `Repository not found` → private, do §3 first.	
Seeding decision	Which reference layers you want on this server (§5.1). Default: base catalogue + GCC extras (automatic), no Ghana pack, **never** the fabricated test portfolio.	
Maintenance window	The PM2 reload is zero-downtime, but migrations may hold locks for a few seconds. Choose a quiet moment.	

Also confirm outbound HTTPS from the server to `github.com` (or SSH port 22 if you use a deploy key): `curl -sSo /dev/null -w '%{http_code}\n' https://github.com` should print `200` (this form also works behind a corporate proxy, where `curl -I` would show the proxy's own reply first).

3. GitHub access to the new repository

> **If `iwadaya/Saudi_re` is public** (the decision taken in September 2026 for simplicity), **skip this section**: anonymous read-only `clone` / `fetch` over HTTPS works with the plain URL `https://github.com/iwadaya/Saudi_re.git`, no token or key is involved, and any credential still stored for `github.com` is simply not used (Git only sends credentials after a 401 challenge). Check with `git ls-remote --heads https://github.com/iwadaya/Saudi_re.git main` from any machine — it prints the commit id without prompting. Keep this section for the day the repository is made private again: the first symptom will be `remote: Repository not found` on fetch, and one of the two options below fixes it. Remember that a public repository exposes the whole codebase and every document in it, so keep secrets, real data and internal hostnames out of commits.

GitHub does not accept account passwords for Git operations, and credentials are scoped to a repository — the token or key that opened `Darchville-Analytics/modelling_tool` will fail against a **private** `iwadaya/Saudi_re` with `remote: Repository not found` or `Permission denied`. For a private repository set up one of the two options below **before** touching the checkout.

3.1 Option 1 — fine-grained personal access token (HTTPS, recommended)

1. On GitHub, signed in as **`iwadaya`** — the account that owns the repository. A fine-grained token can only reach repositories owned by the account (or organisation) that creates it, so a collaborator cannot mint one for `iwadaya/Saudi_re`; a collaborator uses a deploy key (§3.2) or asks the owner for the token (a classic full-scope token stays off the table, as v2.0 §9.2 requires). Go to **Settings** → **Developer settings** → **Personal access tokens** → **Fine-grained tokens** → **Generate new token**.
2. **Resource owner:** `iwadaya`. **Repository access:** *Only select repositories* → `Saudi_re`. **Permissions** → **Repository permissions** → **Contents: Read-only** (Metadata: Read is added automatically). Expiry: 90 days; put the rotation date in your calendar (v2.0 §9.2).
3. On the server, remove the stored token for the old repository so Git prompts for the new one, then test access **interactively once** as the deploy user so the new token is stored and later deploys never prompt:

```
# Remove the old github.com line from the deploy user's credential store (if any).
sudo -u universe -H bash -c 'test -f ~/.git-credentials && sed -i "/github\.com/d" ~/.git-
credentials; git config --global credential.helper store'

# Test read access to the NEW repository. Username = your GitHub username, password = the new
token.
sudo -u universe -H git ls-remote --heads https://github.com/iwadaya/Saudi_re.git main
#   <40-hex commit id>   refs/heads/main        ← success; the token is now saved in
~universe/.git-credentials (mode 600)
```

3.2 Option 2 — SSH deploy key

A GitHub deploy key can be attached to **one** repository only, so the key on `modelling_tool` cannot simply be reused. Generate a second key and give it its own host alias:

```
sudo -u universe -H install -d -m 700 /home/universe/.ssh          # a server that used HTTPS
so far has no .ssh directory yet
sudo -u universe -H ssh-keygen -t ed25519 -C "universe-deploy-saudi_re" -N "" -f
/home/universe/.ssh/id_ed25519_saudi_re
sudo -u universe -H cat /home/universe/.ssh/id_ed25519_saudi_re.pub
#   → GitHub: iwadaya/Saudi_re → Settings → Deploy keys → Add deploy key (leave "Allow write
access" unticked)

sudo -u universe -H tee -a /home/universe/.ssh/config >/dev/null <<'EOF'
Host github.com-saudi_re
  HostName github.com
  IdentityFile /home/universe/.ssh/id_ed25519_saudi_re
  IdentitiesOnly yes
EOF
sudo -u universe -H chmod 600 /home/universe/.ssh/config
sudo -u universe -H ssh -T git@github.com-saudi_re        # "Hi iwadaya/Saudi_re! You've
successfully authenticated…"
```

With this option the remote URL in §4 step 3 is `git@github.com-saudi_re:iwadaya/Saudi_re.git` instead of the HTTPS URL.

4. Path A — switch the existing server to the new repository

All commands assume the v2.0 layout (`/opt/universe` , user `universe` , PM2). Run them from an admin shell with `sudo` ; where a command must run as the application user it is prefixed with `sudo -u universe -H` .

Step 1 — Take a backup and record the starting point

```
cd /opt/universe
sudo -u universe -H git log -1 --format='%h %ci %s'   # currently deployed commit — keep this
for rollback
sudo -u universe -H npm run migrate:status --prefix server | tail -n 3     # must end with
"Pending (0):"

# Full database dump with the checked-in script (same as the nightly job). Prints the dump
path.
sudo install -d -o universe -g universe -m 750 /var/backups/universe      # harmless if it
already exists; universe cannot create it itself
sudo -u universe -H bash -c 'cd /opt/universe && set -a && . ./.env && set +a &&
BACKUP_DIR=/var/backups/universe scripts/backup-db.sh'
ls -lt /var/backups/universe | head -3                # the new .dump file is at the top
```

If `BACKUP_DIR` differs on your server, use the value from your cron entry (v2.0 §4.3). Do not continue without a dump you can see on disk.

Step 2 — Confirm access to the new repository

Make sure `git ls-remote --heads https://github.com/iwadaya/Saudi_re.git main` succeeds **as the `universe` user** without prompting — `deploy/deploy.sh` fetches as that user and cannot answer a password prompt. With the repository public it just works; if it prompts or says `Repository not found` , the repository is private: complete §3 first.

Step 3 — Point the checkout at `iwadaya/Saudi_re`

```
# HTTPS (Option 1). For a deploy key (Option 2) use: git@github.com-
saudi_re:iwadaya/Saudi_re.git
sudo -u universe -H git -C /opt/universe remote set-url origin
https://github.com/iwadaya/Saudi_re.git
sudo -u universe -H git -C /opt/universe remote -v                          # both lines now
show iwadaya/Saudi_re
sudo -u universe -H git -C /opt/universe fetch --prune --tags origin        # replaces the
old origin/* refs

# What you are about to deploy:
sudo -u universe -H git -C /opt/universe log --oneline HEAD..origin/main | wc -l        #
number of new commits
sudo -u universe -H git -C /opt/universe diff --stat HEAD origin/main --
server/src/db/migrations | tail -n 1   # new migration files, if any
sudo -u universe -H git -C /opt/universe merge-base --is-ancestor HEAD origin/main && echo
"OK: fast-forward" || echo "WARNING: deployed commit is not in the new history — read §7.4
(orphan check) before migrating"

# Which migrations the deploy will apply — look for 132 and 143 (they rename/reset the seeded
accounts, see Step 6):
sudo -u universe -H git -C /opt/universe diff --name-only HEAD origin/main --
server/src/db/migrations

# Put the new tree on disk. The old modelling_tool checkout has no deploy/ directory — the
deployment
# kit arrives with this checkout, so it must exist before Step 5 can call it.
sudo -u universe -H git -C /opt/universe checkout -B main origin/main
sudo -u universe -H git -C /opt/universe log -1 --format='%h %ci %s'
ls /opt/universe/deploy/deploy.sh                                           # must exist now
```

Only the source tree on disk has moved to the new commit. The running PM2 workers keep the code they loaded at start, and `client/dist` and `node_modules` are untracked, so users notice nothing until the reload in Step 5.

Step 4 — Check the environment file

The variables from v2.0 remain valid; the new repository needs nothing new. Confirm the critical ones (passwords are masked in the output):

```
sudo -u universe -H bash -c 'cd /opt/universe && ls -l .env && grep -E
"^(NODE_ENV|PORT|DATABASE_URL|CORS_ORIGIN|RUN_MIGRATIONS_ON_BOOT|SEED_ON_DEPLOY|ALLOW_LOCAL_U
PLOADS|ALLOW_DEMO_AUTH)=" .env | sed -E "s#(://[^:]+:)[^@]*@#\1***@#"'
```

Expected: `-rw-------` owned by `universe`; `NODE_ENV=production`; `PORT=4000`; exactly one `DATABASE_URL`; `RUN_MIGRATIONS_ON_BOOT=false` (or absent — the default is false); `ALLOW_DEMO_AUTH` absent; `SEED_ON_DEPLOY` absent, or `reference` if you want the Ghana pack (§5.1). If you want to add `SEED_ON_DEPLOY=reference`, do it now — the deploy step reads it.

Step 5 — Deploy from the new repository

Option A1 — the repository's deploy script (recommended). It does every step in order, stops at the first failure, and runs the Git/npm/build/migrate/seed steps as the `universe` user. On a PM2 server pass the env-file location and skip the systemd restart:

```
sudo env ENV_FILE=/opt/universe/.env /opt/universe/deploy/deploy.sh --ref main --no-restart
```

What it prints, in order (each step is a `►` header): `1. fetching origin and checking out 'main'` (and the commit it landed on) → `2. installing dependencies from lockfiles (npm ci)` → `3. building the client bundle` → `4. database migrations` (the applied/pending list, then one `migration completed` line per file and `migrations complete`) → `5. reference data (countries, currencies, brokers, cedants, …) — never contracts` (the counts block shown in §5.2, then the `seed-on-deploy:` line) → `6. restart skipped (--no-restart) — run: sudo systemctl restart universe && deploy/smoke-test.sh` → `done — <commit> is deployed`. **Ignore the `systemctl` hint in step 6 on a PM2 server** — reload PM2 as shown next. Normal noise between the headers: a dotenv banner (`◇ injected env (N) from ../.env`) before every Node step, JSON `DB pool connecting` / `DB pool configured` lines, and inside step 5 an early `reference data ready … "cedants":22` line *before* the counts block reports 40 — the block is the authoritative result. The migrations and the seed run **before** the reload, so a failure leaves the old version running.

Now reload the application under PM2 — as the user that owns the PM2 daemon (`universe` per v2.0), with the same `pm2` you started it with, and with the same environment overrides (`PM2_INSTANCES`, `DB_POOL_MAX`) exported if you used any at start, because `ecosystem.config.cjs` re-derives the worker count and per-worker pool on every reload — and smoke-test it:

```
sudo -u universe -H bash -c 'cd /opt/universe && pm2 reload ecosystem.config.cjs && pm2 save'
# zero-downtime rolling reload (= npm run cluster:reload)
sudo -u universe -H pm2 status
# every universe worker "online", uptime just reset
BASE_URL=http://127.0.0.1:4000 /opt/universe/deploy/smoke-test.sh
# must end with PASSED
```

Option A2 — the same steps by hand (when each step must be inspected individually):

```
sudo -u universe -H bash                      # open a shell as the application user
cd /opt/universe
git fetch --prune --tags origin
git checkout -B main origin/main
git log -1 --format='%h %ci %s'
npm ci --include=dev && npm ci --include=dev --prefix client && npm ci --prefix server   # --
include=dev: the client build needs vite even with NODE_ENV=production
npm run build --prefix client
npm run migrate:status --prefix server        # review the pending list before applying it
npm run migrate:up --prefix server            # apply BEFORE reloading the app
node server/src/db/seeds/run.js               # reference data — prints the counts block
(§5.2); aborts if any business table changed
npm run seed:deploy --prefix server           # Ghana pack if SEED_ON_DEPLOY=reference;
otherwise prints "skipping" (expected)
pm2 reload ecosystem.config.cjs && pm2 save   # zero-downtime PM2 reload (= npm run
cluster:reload). Do NOT source .env in this shell first — see §7.6 P1
exit
BASE_URL=http://127.0.0.1:4000 /opt/universe/deploy/smoke-test.sh
```

On a systemd (deployment-kit) server the whole step is simply `sudo /opt/universe/deploy/deploy.sh --ref main` — it reads `/etc/universe/universe.env`, restarts the service and runs the smoke test itself.

Step 6 — Check the accounts the migrations may have changed

Two migrations touch user accounts, and they run against **any** database that has not seen them yet — including one that already holds real users:

- **132** renames the v2.0-era seeded accounts `cuo`, `underwriter`, `edwin.taruvinga`, `catho.ba` and `chongo.nkalamo` to `chief.underwriter` and `underwriter1` – `4`, **resets their passwords to `demo2026`**, re-activates them, clears any forced password change, and puts `chongo.nkalamo` (the v2.0

Chief Actuary login) on the Underwriter role with a 25 M mandate. Rotated passwords on those five accounts are lost. Accounts with any other username are not touched.

- **143** inserts `retro.manager` (password `demo2026`) if no account with that username exists.

Step 3 showed whether these files were pending. Whether or not they were, do this now, before users are told:

```
cd /opt/universe/server
sudo -u universe -H node scripts/manage-user.js list                        # who exists now,
and who is active
sudo -u universe -H node scripts/manage-user.js set-password chief.underwriter        #
prompts for a real password
sudo -u universe -H node scripts/manage-user.js deactivate underwriter1               # …2,
3, 4 and retro.manager — or set-password <user> --must-change to hand them to real people
```

Every `manage-user` action revokes the account's live sessions and writes an `audit_log` row (actor `OPS_CLI`). If the v2.0 Chief Actuary used `chongo.nkalamo`, that login is now `underwriter4`: give the person a fresh account from the ⚙ **USERS** screen (or re-point `underwriter4` with `set-password --must-change`).

Step 7 — Verify

Run the checklist in §7. At minimum, before you tell users: `Pending (0)` from `migrate:status`, `PASSED` from the smoke test, `db.ok:true` from `/api/health/deep` **through nginx** (`curl -fsS https://<APP_DOMAIN>/api/health/deep`), the seed counts from `npm run seed:reference:check`, the user list check (R6), and one real login over HTTPS.

Step 8 — After the switch

- `pm2 save` was run (step 5), so the process list survives a reboot; `pm2 startup` from v2.0 is unchanged.
- If you will run the weekly restore drill (`scripts/verify-restore.sh`) with the app's `DATABASE_URL`, grant the role the right to create its scratch database once: `sudo -u postgres psql -c 'ALTER ROLE universe CREATEDB'`.
- The backup cron, nginx site and TLS certificate reference paths, not the repository — nothing to change. Run `scripts/backup-db.sh` once more now so the newest dump reflects the migrated schema.
- Delete or let expire the old token/deploy key for `modelling_tool`; if you created a new token for a private `Saudi_re`, record its expiry.
- Add a line to your deployment log: date, old commit → new commit, migrations applied (from step 5 output), seed counts, who signed off (Appendix B).

Rollback

The previous commit is still in the local repository and in the new repository's history. It predates the deployment kit, so roll back by hand rather than with `deploy/deploy.sh` (checking the old commit out removes `deploy/` again):

```
sudo -u universe -H bash
cd /opt/universe
git checkout -q <old commit from step 1>
npm ci --include=dev && npm ci --include=dev --prefix client && npm ci --prefix server
npm run build --prefix client
pm2 reload ecosystem.config.cjs && pm2 save
exit
```

Migrations are forward-only. If a migration itself is the problem, stop the app (`sudo -u universe -H pm2 stop universe`), restore the step-1 dump, then roll the code back as above. The dump was taken with `--no-`

`owner`, so restore it **with `--role=universe`** — without it every restored table ends up owned by `postgres` and the application gets `permission denied for table …`:

```
sudo -u postgres pg_restore --clean --if-exists --no-owner --role=universe -d universe
/var/backups/universe/<file>.dump
```

To move forward again after a rollback, check `main` out first (`sudo -u universe -H git -C /opt/universe checkout -B main origin/main`) so the kit is back on disk, then repeat Step 5.

5. Seeding the database

5.1 What gets seeded, by whom, and when

"Seeding" is four separate layers. Only the first three are ever appropriate on this server.

Layer	What it writes	When it runs	Writes contracts?
Migrations (`npm run migrate:up`)	The schema, and the base catalogue that ships inside it: **75 countries** with regions (migration 045 — GCC, Levant, North Africa, Sub-Saharan Africa incl. Ghana, Europe, Americas, South/Southeast/East Asia; not every country in the world, add others on the reference screens), 32 currencies each with a USD rate, 11 brokers, 15 core reinsurers and 22 cedants (005), inflation (CPI) series for Saudi Arabia, the UAE and the UK (2000–2026), the CRESTA-zone table (empty until zones are added), facultative reference tables and taxonomies, 11 treaty types, 15 classes of business (the 15th, *Political Violence*, from 123), roles and the mandate hierarchy, and the demo personas (`chief.underwriter`, `retro.manager`, `underwriter1`–`4`, password `demo2026`) — **132 and 143 also rename and reset existing accounts, see §4 Step 6**.	Deploy step 4. Each file runs once, recorded in `public._migrations`.	No
Boot seed (`ensureReferenceData`)	Re-asserts its hard-coded canonical lists (45 countries, 32 currencies, 11 brokers, 11 treaty types, 14 classes, 15 reinsurers, 22 cedants) with `WHERE NOT EXISTS` — on a migrated database this is normally a no-op.	Every application start, and inside the reference seed.	No

Layer	What it writes	When it runs	Writes contracts?
Reference seed (`node server/src/db/seeds/run.js`, `npm run seed:reference`)	The boot seed plus `002_reference_data.sql`: the canonical mirror and the **GCC/MENA market extras**. Net effect on a migrated database: **+18 cedants** — MEDGULF, SALAMA, Oman Insurance Co, Sukoon Insurance, GIG Bahrain, Solidarity Bahrain, Kuwait Insurance Co, Dhofar Insurance, National Life & General, SAICO, Allianz Saudi Fransi, AXA Cooperative Insurance, Dubai National Insurance, Al Ahleia Insurance, Qatar Insurance Company, Doha Insurance Group, Jordan Insurance Company, Misr Life Insurance — and **+10 reinsurers** — Saudi Re, Kuwait Re, Arab Re, Gulf Re, Milli Re, GIC Re, Malaysian Re, RenaissanceRe, Berkshire Hathaway Re, AXA XL Re. Idempotent (upserts on the natural keys from migration 150; see the soft-delete caveat in §5.3).	Deploy step 5 in `deploy/deploy.sh` (skip with `--no-seed`), or by hand at any time.	**No** — it counts `contract`, `quote`, `claim` and `uw_user` before and after and **exits non-zero if any of them changed**.
Ghana pack (`npm run seed:deploy` with `SEED_ON_DEPLOY=reference`)	Ghana country metadata, the `GHS` currency and its USD rate, the Ghana CPI series 2000–2026, 8 Ghana CRESTA zones, 8 Ghanaian cedants. Idempotent.	Deploy step 5 (after the reference seed), only while `SEED_ON_DEPLOY=reference` is in the env file. Otherwise the hook prints `seed-on-deploy: SEED_ON_DEPLOY is not set — skipping.`	No
Test portfolio (`SEED_ON_DEPLOY=1` / `reset`, or `npm run seed:treaties`)	100 **fabricated** treaties with full pricing, triangles, workflow and audit rows.	Only when someone sets those values.	**YES — test/demo databases only. Never on this server.**

Decision for this server. Leave `SEED_ON_DEPLOY` unset unless the Ghana lookups are wanted; if they are, add exactly one line `SEED_ON_DEPLOY=reference` to `/opt/universe/.env` (no restart needed — the deploy hook reads the file). Never set `1` or `reset` here. If fabricated treaties ever appear, remove only them with `node server/scripts/seedTestTreaties.js --reset-only` (deletes rows stamped `seed:test-treaties`, nothing else).

5.2 What "seeded correctly" looks like

Verified against a freshly migrated PostgreSQL 16 database at this commit. The reference seed prints this block; `npm run seed:reference:check` prints the same counts read-only under the headings `Reference data (read-only check):` and `Business data (never written by this script):`.

```
Reference data after seeding:
  countries            75
  currencies           32
  brokers              11
  reinsurers           25
  cedants              40
  treaty_types         11
  classes_of_business  15
  roles                9
Business data (unchanged):
  contracts  0
  quotes     0
  claims     0
  users      6
```

On your existing database expect **at least** these reference counts: reinsurers rise from 15 to 25 and cedants from 22 to 40 the first time the reference seed runs (the GCC extras), the Ghana pack adds 1 currency and 8 cedants, anything users have created on the reference screens adds to the totals, and the check counts **deactivated rows too** (soft-deleted or superseded entries stay in the tables). Only a fresh database shows the exact block above. The business rows (`contracts`, `quotes`, `claims`, `users`) must show **your real numbers, unchanged** by the seed — that is the invariant the script enforces.

Spot checks that prove the rows landed in *this* database and are visible to the app. The four Saudi names MEDGULF, SALAMA, SAICO, Allianz Saudi Fransi and AXA Cooperative Insurance exist **only** because of the reference seed (the others were already there from the migrations), so they are the ones to look for:

```
cd /opt/universe && set -a && . ./.env && set +a
psql "$DATABASE_URL" -c "SELECT company_name FROM public.companies c JOIN public.country k
USING (country_id) WHERE k.country_code='SA' AND c.is_active IS NOT FALSE ORDER BY 1"
#   AXA Cooperative Insurance, Al Rajhi Takaful, Allianz Saudi Fransi, Bupa Arabia, Gulf
Union Insurance, MEDGULF,
#   Malath Insurance, SALAMA, Saudi Arabian Cooperative Insurance (SAICO), Tawuniya, Walaa
Insurance   (11 rows on a fresh database)
psql "$DATABASE_URL" -tAc "SELECT count(*) FROM public.reinsurers WHERE reinsurer_name IN
('Saudi Re','Kuwait Re','Arab Re','Gulf Re')"   # 4
psql "$DATABASE_URL" -tAc "SELECT count(*) FROM public.contract WHERE import_metadata-
>>'source'='seed:test-treaties'"   # 0 — no fabricated treaties
psql "$DATABASE_URL" -tAc "SELECT count(*) FROM public.currency WHERE currency_code='GHS'"
# 1 only if the Ghana pack was requested
```

In the browser: open a new treaty — the **Cedant** dropdown lists the Saudi companies above (including MEDGULF and SAICO), the **Currency** dropdown includes SAR/AED/KWD, and the **Reinsurer** panel includes Saudi Re, Kuwait Re and Arab Re.

5.3 Re-running the seed, and what can go wrong

- The seed is idempotent: run `sudo -u universe -H node server/src/db/seeds/run.js` from `/opt/universe` as often as you like. One caveat: the natural keys from migration 150 cover **active** rows only, so a seeded country, class of business, broker or cedant that a user has *deactivated* on the reference screens is re-created as a new active row by the next seed (the deactivated row stays; counts grow by one). If a seeded name must stay hidden, rename it instead of deactivating it. Currencies, treaty types and reinsurers are not affected.
- It must run **after** `migrate:up`. On a database that is behind migration 150 it fails inside `002_reference_data.sql` — with `column "is_active" does not exist` if it is behind 141, or `there is no unique or exclusion constraint matching the ON CONFLICT specification` if it is at 141–149 — apply migrations, then re-run.

- If it aborts with `seed:reference changed business table "…"`, something other than reference data moved during the run (most likely a user working at that moment). Nothing is rolled back that users did; re-run at a quiet moment and compare the two counts.
- A seed failure inside `deploy/deploy.sh` stops the script before the reload — the old version keeps running. Fix, then re-run the deploy.

6. Path B — fresh server from the new repository (deployment kit)

Use this when you want a clean box rather than switching in place. The kit is documented step by step in `deploy/README.md` (systemd + nginx + PostgreSQL 16, reference data only). The outline, with the data move from the old server:

B1 — Prepare the host. Ubuntu 22.04/24.04, a sudo user, DNS for `APP_DOMAIN`, the TLS certificate at hand. The kit already points at the new repository; while the repository is public the clone needs no credentials (if it is private, it prompts for your GitHub username and the fine-grained token, §3.1):

```
git clone https://github.com/iwadaya/Saudi_re.git /tmp/universe-kit
sudo APP_DOMAIN=universe.example.internal REPO_URL=https://github.com/iwadaya/Saudi_re.git
GIT_REF=main \
     bash /tmp/universe-kit/deploy/install-server.sh
```

It installs Node 20, PostgreSQL 16 and nginx, creates the `universe` user, clones the repository into `/opt/universe`, creates the database role and database, writes `/etc/universe/universe.env` with generated secrets and the ready `DATABASE_URL`, and installs the systemd unit and nginx site. Review the env file (`sudo -e /etc/universe/universe.env`), in particular `CORS_ORIGIN`. If the repository is private, store the token for the deploy user once so later `deploy.sh` runs never prompt: `sudo -u universe -H git config --global credential.helper store && sudo -u universe -H git -C /opt/universe ls-remote --heads origin main` (enter username + token).

B2 — Move the data (optional; skip for an empty pilot database). On the old server take a fresh dump (`scripts/backup-db.sh`), copy the `.dump` file and the uploads directory (`/opt/universe/uploads`, or your `UPLOAD_DIR`) across, then on the new server restore **before** the first deploy so the migrations bring the restored schema forward:

```
sudo systemctl stop universe 2>/dev/null || true
sudo -u postgres pg_restore --clean --if-exists --no-owner --role=universe -d universe
/path/to/universe-<stamp>.dump
sudo rsync -a --chown=universe:universe old-server:/opt/universe/uploads/
/var/lib/universe/uploads/
```

B3 — Build, migrate, seed, start, smoke-test in one command:

```
sudo /opt/universe/deploy/deploy.sh --ref main
```

The expected tail of the seed output is the block in §5.2 (with your real business counts if you restored data), followed by `systemctl restart universe` and the smoke test ending in `PASSED`.

B4 — Install the real TLS certificate (`deploy/README.md` step 7), then run the §7 checklist through nginx: `BASE_URL=https://<APP_DOMAIN> /opt/universe/deploy/smoke-test.sh`. Use the kit variants noted at the top of §7 for the rows that mention PM2 or the env-file location.

B5 — Cut over. Point DNS at the new server, make sure `CORS_ORIGIN` is exactly the URL users type, restart (`sudo systemctl restart universe`), install the backup cron (`deploy/README.md` step 10), and decommission the old server after the retention period.

Accounts. On a fresh database the migrations create the demo personas (`demo2026`). On a *restored* database the pending migrations may still add or rename accounts — 143 adds `retro.manager` if absent, and 132 renames `cuo` / `underwriter` / `edwin.taruvinga` / `catho.ba` / `chongo.nkalamo` to `chief.underwriter` / `underwriter1–4` and resets their passwords to `demo2026` (§4 Step 6). Either way, run `node scripts/manage-user.js list` after B3 and rotate or deactivate every persona before sharing the URL (`deploy/README.md` step 9).

7. Checklist — is the database linked to the app, and does it align?

Work through the groups in order. "Command" runs on the application server; `psql` commands need `DATABASE_URL` in the shell — load it once with `cd /opt/universe && set -a && . ./.env && set +a` (as `universe` or root) **in a shell you will not use for `pm2` commands** (see P1). Tick every row before sign-off.

> **Deployment-kit (systemd) server?** Use these variants: C1 → `ls -lL /opt/universe/.env` = `-rw-r-- --- root universe` (a symlink to `/etc/universe/universe.env`); C6, R1, R2 → `sudo journalctl -u universe --no-pager -n 300 | grep …` instead of `pm2 logs`; P1 → one process, `pool.max` 50 unless `DB_POOL_MAX` is set in the env file; P2 unchanged.

7.1 Configuration — the app is told about exactly one database

#	Check	Command	Expected	Proves
C1	One env file, locked down	`ls -l /opt/universe/.env`	`-rw------- universe universe`	Secrets readable by the app user only
C2	Exactly one `DATABASE_URL`	`grep -c '^DATABASE_URL=' /opt/universe/.env`	`1`	No duplicate/overridden connection string
C3	No placeholders	`grep -c CHANGE_ME /opt/universe/.env`	`0` (grep exits 1 when the count is 0 — expected)	Real values everywhere
C4	Production mode, migrations explicit	`grep -e '^NODE_ENV=' -e '^RUN_MIGRATIONS_ON_BOOT=' /opt/universe/.env`	`NODE_ENV=production`, `RUN_MIGRATIONS_ON_BOOT=false` (or absent)	Schema changes only happen in the deploy step
C5	Seeding posture	`grep '^SEED_ON_DEPLOY=' /opt/universe/.env`	no output (absent; grep exits 1), or `SEED_ON_DEPLOY=reference` — never `1` / `reset`	No fabricated treaties can be seeded
C6	The app loaded *that* file	`sudo -u universe -H pm2 logs universe --lines 300 --nostream 2>/dev/null | grep 'startup configuration loaded' | tail -n 1`	newest line (format `0|universe | <date>: {json}`) with `"envFile":"/opt/universe/.env"`	The running process read this env file, not another

7.2 Connectivity — the database answers with the app's own credentials

#	Check	Command	Expected	Proves
D1	Connect with the app's URL	`psql "$DATABASE_URL" -c 'select 1'`	one row, `1`	Host, port, role, password and database name are right
D2	Identify what you are connected to	`psql "$DATABASE_URL" -tAc "select current_database(), current_user, inet_server_addr(), inet_server_port(), version()"`	`universe|universe|127.0.0.1|5432|PostgreSQL 16…` (your values)	You are looking at the intended database, not a stray one
D3	Postgres is up and listening locally	`sudo systemctl status postgresql --no-pager | head -3` and `ss -tlnp | grep 5432` (alternatives: `pg_lsclusters`; `psql "$DATABASE_URL" -tAc "show listen_addresses"`)	`active (running)`; only loopback sockets — `127.0.0.1:5432` and/or `[::1]:5432`, never `0.0.0.0:5432`, `*:5432` or `[::]:5432` (`pg_lsclusters` → `online`; `listen_addresses` → `localhost`)	Database service healthy and not exposed
D4	Role owns the schema	`psql "$DATABASE_URL" -tAc "select pg_get_userbyid(datdba) from pg_database where datname=current_database()"`	`universe`	Migrations can create/alter objects without superuser help

7.3 Runtime — the *running* application is connected to *that* database

#	Check	Command	Expected	Proves
R1	Boot log names the database	`sudo -u universe -H pm2 logs universe --lines 300 --nostream 2>/dev/null | grep 'DB pool connecting' | tail -n 1`	`"url":"postgresql://universe:***@localhost:5432/universe"` matching `DATABASE_URL` (password masked)	The process is using the same host/port/db you tested in D1–D2

#	Check	Command	Expected	Proves
R2	Boot sequence completed	same log, `grep -e 'database connection verified' -e 'migrations skipped' -e 'migrations complete' -e 'reference data ready' -e 'http server listening' | tail -n 12`	For the most recent start, once **per worker**: `database connection verified`, `migrations skipped`, `reference data ready` (appears twice per worker), `http server listening`. `migrations skipped` is correct — the deploy applied them; `migrations complete` appears instead only if `RUN_MIGRATIONS_ON_BOOT=true`	App verified the DB, found the schema, seeded, and is serving
R3	Deep health via the app port	`curl -sS -D - http://127.0.0.1:4000/api/health/deep`	HTTP 200, `"db":{"ok":true,"pingMs":<small>,"pool":{…,"max":<per-worker pool>,…}}`, header `X-Pool-Waiting: 0`. Under `ecosystem.config.cjs` the per-worker pool is `floor(80 / workers)` — 20 on a 4-worker box, 40 with 2 workers; 50 only when `DB_POOL_MAX` is set or the app runs outside the ecosystem file (see P1)	Live round-trip from the app to the DB, pool not saturated
R4	Deep health through nginx/TLS	`curl -sS https://<APP_DOMAIN>/api/health/deep`	same body, `"env":"production"`	Users' path reaches the same connected app
R5	DB sees the app's connections	`psql "$DATABASE_URL" -c "select usename, client_addr, state, count(*) from pg_stat_activity where datname=current_database() group by 1,2,3"`	rows with `usename = universe` (idle/active) from the app host	The connections exist in Postgres, from the expected role and host

#	Check	Command	Expected	Proves
R6	Read path: app data = DB data	`curl -sS http://127.0.0.1:4000/api/auth/users | grep -o '"username":"[^"]*"' | cut -d'"' -f4 | sort` vs `psql "$DATABASE_URL" -tAc "select username from public.uw_user where is_active order by 1"`	identical lists with the same row count. Beware the static fallback the API serves when `uw_user` is empty or missing: exactly two rows, `chief.underwriter` and `underwriter1`, with `user_id` `00000000-…-0001` / `-0002` — the same names a migrated database has, so compare the **count** with `select count(*) from public.uw_user where is_active`	The login screen is served from this database
R7	Write path: a login lands in the audit log	Log in once in the browser, then `psql "$DATABASE_URL" -c "select event_type, payload->'actor'->>'name' as username, actor as user_id, created_at from public.audit_log where event_type='LOGIN' order by created_at desc limit 3"`	top row: `username` = the account you just used (the `actor` column holds its user id), `created_at` = just now	The app writes to this database, and the clock/timezone are sane
R8	Smoke test	`BASE_URL=https://<APP_DOMAIN> /opt/universe/deploy/smoke-test.sh` (add `SMOKE_USER` / `SMOKE_PASSWORD` to test a real login)	`PASSED`	R3–R4 plus the SPA in one exit code. Its user-list step only checks that the list is non-empty, so it does **not** replace R6

7.4 Schema alignment — the code and the database agree on the migrations

#	Check	Command	Expected	Proves
S1	Nothing pending	`cd /opt/universe && sudo -u universe -H npm run migrate:status --prefix server | tail -n 3`	`Pending (0):`	Every migration file in this checkout has been applied
S2	Counts match	`psql "$DATABASE_URL" -tAc "select count(*) from public._migrations"` vs `ls /opt/universe/server/src/db/migrations/*.sql | wc -l`	equal (**149** at this commit)	No file applied twice, none missing

#	Check	Command	Expected	Proves
S3	No orphans (DB ahead of code)	`psql "$DATABASE_URL" -tAc "select filename from public._migrations" | sort > /tmp/applied.txt; ls /opt/universe/server/src/db/migrations | sort > /tmp/ondisk.txt; comm -23 /tmp/applied.txt /tmp/ondisk.txt`	**empty**	The database was never migrated by code this checkout does not contain. If a filename prints, the old repository shipped a migration the new one lacks — stop and reconcile before deploying further (`docs/migration-audit.md`)
S4	No stragglers (code ahead of DB)	`comm -13 /tmp/applied.txt /tmp/ondisk.txt`	**empty**	Same as S1, file-by-file
S5	Last migration is recent and expected	`psql "$DATABASE_URL" -c "select filename, applied_at from public._migrations order by applied_at desc limit 5"`	newest = `157_quote_offer_status_checks.sql` at this commit, `applied_at` = your deploy time	The deploy's migration step ran against this database
S6	Core tables answer	`psql "$DATABASE_URL" -c "select (select count(*) from public.contract) contracts, (select count(*) from public.quote) quotes, (select count(*) from public.claim) claims, (select count(*) from public.uw_user) users"`	your real business counts, unchanged from before the switch	The data survived; the app's tables exist

7.5 Reference-data alignment — the lookups the app expects are present

#	Check	Command	Expected	Proves
F1	Reference counts	`cd /opt/universe && sudo -u universe -H npm run seed:reference:check`	headings `Reference data (read-only check):` / `Business data (never written by this script):` ; countries ≥75, currencies ≥32, brokers ≥11, reinsurers ≥25, cedants ≥40, treaty types ≥11, classes ≥15, roles ≥9 (the check counts deactivated rows too; only a fresh database shows the exact §5.2 numbers); business rows = yours, unchanged	The reference seed reached this database

#	Check	Command	Expected	Proves
F2	Seed-only cedants and reinsurers visible	SQL in §5.2	MEDGULF, SALAMA, SAICO, Allianz Saudi Fransi and AXA Cooperative Insurance listed; reinsurer count query = 4	GCC/MENA extras present — these names come only from the reference seed
F3	No fabricated treaties	`psql "$DATABASE_URL" -tAc "select count(*) from public.contract where import_metadata->>'source'='seed:test-treaties'"`	`0`	The test portfolio was never loaded
F4	UI dropdowns	New treaty screen: Cedant, Currency, Reinsurer, Class of business, Treaty type	populated as in §5.2	The app reads the seeded lookups

7.6 Capacity and users

#	Check	Command	Expected	Proves
P1	Connection budget	`psql "$DATABASE_URL" -tAc "show max_connections"` ; PM2 workers from `pm2 status` ; per-worker pool from R3 `pool.max` or `sudo -u universe -H pm2 env 0 | grep DB_POOL_MAX`	workers × pool.max ≤ max_connections − 20 (e.g. 4 × 20 = 80 ≤ 80). Under PM2 the per-worker pool is what `ecosystem.config.cjs` injects: `floor(80 / workers)` (min 8), **unless `DB_POOL_MAX` was exported in the shell that ran `pm2 start` / `pm2 reload`** — then that value is used verbatim per worker (4 × 50 = 200 > 100). A `DB_POOL_MAX` line in `.env` does **not** reach the PM2 workers (they already carry the injected value); it only affects CLI scripts and the single-process systemd path. So never run `pm2 reload` from a shell that has sourced `.env` ; if in doubt reload with `env -u DB_POOL_MAX pm2 reload ecosystem.config.cjs`	The cluster cannot exhaust Postgres

#	Check	Command	Expected	Proves
P2	Accounts	`cd /opt/universe/server && sudo -u universe -H node scripts/manage-user.js list`	your real users active; seeded personas (`chief.underwriter`, `retro.manager`, `underwriter1–4`) rotated or deactivated	No `demo2026` login remains
P3	Backup after the switch	`ls -lt /var/backups/universe | head -2`; weekly `scripts/verify-restore.sh` exit 0	a dump newer than the deploy. `verify-restore.sh` creates and drops a scratch database, so the role in its `DATABASE_URL` needs `CREATEDB` — grant it once with `sudo -u postgres psql -c 'ALTER ROLE universe CREATEDB'` (as `deploy/README.md` step 10 does), or run it with a superuser URL; otherwise it fails with `permission denied to create database`	Recovery point reflects the migrated schema

7.7 Sign-off summary

- ☐ §7.1 C1–C6: one env file, one `DATABASE_URL`, production mode, seeding posture correct, app loaded that file.
- ☐ §7.2 D1–D4: `psql` with the app's URL works and identifies the intended database; role owns it.
- ☐ §7.3 R1–R8: boot log, deep health (direct and via nginx), `pg_stat_activity`, user list = `uw_user`, login audited, smoke test `PASSED`.
- ☐ §7.4 S1–S6: `Pending (0)`, counts equal, no orphans, no stragglers, last migration as expected, business counts unchanged.
- ☐ §7.5 F1–F4: reference counts as in §5.2, GCC cedants visible, zero fabricated treaties, dropdowns populated.
- ☐ §7.6 P1–P3: connection budget, accounts rotated, fresh backup.

8. Troubleshooting the switch

Symptom	Cause	Fix
`remote: Repository not found` / `fatal: Authentication failed` on fetch or `ls-remote`	The repository is (or has become) private and no valid credential is stored: the old token (scoped to `modelling_tool`) is still in `~universe/.git-credentials`, or the new token's resource owner/repository is wrong. A public repository never produces this for a read.	Remove the `github.com` line from the credential file (§3.1), mint the token with resource owner **iwadaya** and repository **Saudi_re**, Contents: Read; retry `git ls-remote` interactively as `universe`

Symptom	Cause	Fix
`Permission denied (publickey)`	Deploy key not added to `Saudi_re`, or the SSH alias/IdentityFile is wrong	§3.2; a deploy key works for one repository only — generate a new key for `Saudi_re`
`remote: Invalid username or token. Password authentication is not supported`	An account password was typed at the prompt	Paste the fine-grained token as the password (v2.0 §12.1)
`deploy.sh` stops at `DATABASE_URL is not set (create /etc/universe/universe.env …)`, usually after `! … not readable`	The script defaults to the kit's env file; your v2.0 server keeps it at `/opt/universe/.env`. Also what you see when the script is started **without `sudo`**: the env file is unreadable to your admin user	Run it as `sudo env ENV_FILE=/opt/universe/.env /opt/universe/deploy/deploy.sh --ref main --no-restart` (it drops to `universe` for the Git/npm/build/migrate/seed steps itself)
`deploy.sh` stops at `run as root (sudo) or as universe`	Started as your admin user without `sudo`, with `DATABASE_URL` already exported in that shell	Prefix with `sudo`
`sudo env ENV_FILE=… deploy/deploy.sh` → `No such file or directory`	The checkout is still the old `modelling_tool` tree, which has no `deploy/` directory	Step 3: `sudo -u universe -H git -C /opt/universe checkout -B main origin/main` puts the kit on disk, then retry
`Failed to restart universe.service: Unit universe.service not found`	`deploy.sh` was run without `--no-restart` on a PM2 server	Nothing is broken — migrations and seed already ran. Reload PM2: `sudo -u universe -H bash -c 'cd /opt/universe && pm2 reload ecosystem.config.cjs'`
`sudo -u universe -H pm2 …` → `pm2: command not found`	PM2 is not installed globally for that user (only as the repository's devDependency)	Use `/opt/universe/node_modules/.bin/pm2` in place of `pm2`, or install it globally as v2.0 §3.1 did (`sudo npm install -g pm2`)
`scripts/verify-restore.sh` fails with `permission denied to create database`	The `universe` role cannot create the scratch database the restore drill uses	`sudo -u postgres psql -c 'ALTER ROLE universe CREATEDB'` once, or run the drill with a superuser `DATABASE_URL` (§7.6 P3)
`deploy.sh` errors while reading the env file (`command not found`, `unexpected token`)	A value in `.env` contains shell-special characters (`&`, spaces, `$`) and the script sources the file with `set -e`, so it dies before anything else runs	Quote that value in `.env` (`KEY='value'` — dotenv and systemd both strip the quotes). As a last resort run with `ENV_FILE=/dev/null` and export `DATABASE_URL` and `PORT` yourself
`vite: not found` during the client build	`npm ci` ran with `NODE_ENV=production` exported and skipped devDependencies	Use `deploy.sh` (it passes `--include=dev`) or `npm ci --include=dev --prefix client`
`migrate:status` still shows pending files after the deploy	Migration step failed earlier (read its error), or it ran against a different `DATABASE_URL` than the app uses	Fix the cause, `npm run migrate:up --prefix server`, then §7.4 S1–S4

Symptom	Cause	Fix
§7.4 S3 prints a filename (orphan migration)	The database was migrated by a version of the old repository that shipped a file this checkout lacks	Do not "fix" the `_migrations` table. Compare the old checkout's `server/src/db/migrations/` with the new one, port the missing file forward as a new additive migration, and follow `docs/migration-audit.md`
Seed fails: `no unique or exclusion constraint matching the ON CONFLICT specification`	Migration 150 not applied yet	Run `migrate:up` first; the seed must follow the migrations
Seed aborts: `seed:reference changed business table`	A business row was written by a user during the run	Re-run at a quiet moment; the seed itself never writes those tables
`seed-on-deploy: SEED_ON_DEPLOY is not set — skipping.`	Expected unless you wanted the Ghana pack	Add `SEED_ON_DEPLOY=reference` to `.env` and re-run `npm run seed:deploy --prefix server`
Fabricated treaties appeared on the dashboards	Someone set `SEED_ON_DEPLOY=1` / `reset` or ran `seed:treaties`	`node server/scripts/seedTestTreaties.js --reset-only` removes only rows stamped `seed:test-treaties`; remove the variable
Login screen lists exactly two users, *Chief Underwriter* and *Underwriter 1*, and `curl -s localhost:4000/api/auth/users` shows `user_id` `00000000-0000-0000-0000-000000000001` / `…0002`	The auth route served its static fallback because `uw_user` is empty or missing — migrations did not run, or the app points at an empty database	§7.3 R1 + §7.4 S1; fix `DATABASE_URL` or run `migrate:up`, reload
`/api/health/deep` returns 503 `db.ok` false	Postgres down, wrong `DATABASE_URL`, or pool exhausted	§7.2 D1–D3; `X-Pool-Waiting` > 0 sustained → raise `DB_POOL_MAX` within §7.6 P1
Postgres logs / app errors `FATAL: sorry, too many clients already`	workers × per-worker pool exceeds `max_connections` — typically because `DB_POOL_MAX=50` was exported in the shell (e.g. after sourcing `.env`) when `pm2 start` / `pm2 reload` ran, so every worker got 50	Reload from a clean shell: `sudo -u universe -H bash -c 'cd /opt/universe && env -u DB_POOL_MAX pm2 reload ecosystem.config.cjs && pm2 save'`, then confirm `pool.max` in `/api/health/deep` (§7.6 P1)
PM2 prints `In-memory PM2 is out-of-date` / version mismatch after the deploy	The repository's `npm ci` installed a local `pm2` that differs from the daemon's version	Keep using the `pm2` you started the app with. If you want them aligned, `pm2 update` (as the PM2 user) saves the list, kills the daemon and resurrects the workers — expect a few seconds of downtime, so do it in the maintenance window and follow with `pm2 save`
App starts with `AUTH_JWT_SECRET is required` / `must not be a placeholder`	Env file incomplete	v2.0 §5.1 — set a ≥32-char secret, reload
Page looks unchanged after the deploy	Browser cache	Hard refresh (Ctrl/Cmd-Shift-R); confirm `git log -1` and PM2 uptime

Appendix A — Quick command reference (v2.0 layout, PM2)

Task	Command
Switch remote	`sudo -u universe -H git -C /opt/universe remote set-url origin https://github.com/iwadaya/Saudi_re.git`
Full deploy, keep PM2	`sudo env ENV_FILE=/opt/universe/.env /opt/universe/deploy/deploy.sh --ref main --no-restart`
Reload app (zero downtime)	`sudo -u universe -H bash -c 'cd /opt/universe && pm2 reload ecosystem.config.cjs && pm2 save'`
Smoke test	`BASE_URL=http://127.0.0.1:4000 /opt/universe/deploy/smoke-test.sh`
Migration status / apply	`npm run migrate:status --prefix server` / `npm run migrate:up --prefix server` (from `/opt/universe`, as `universe`)
Reference seed / check	`node server/src/db/seeds/run.js` / `npm run seed:reference:check`
Ghana pack (optional)	`SEED_ON_DEPLOY=reference` in `.env`, then `npm run seed:deploy --prefix server`
Remove fabricated treaties	`node server/scripts/seedTestTreaties.js --reset-only`
Users	`cd server && node scripts/manage-user.js list | set-password <user> [--must-change] | deactivate <user>`
Health	`curl -s localhost:4000/api/health/deep`
Logs	`sudo -u universe -H pm2 logs universe --lines 100`
Backup / verify	`scripts/backup-db.sh` / `scripts/verify-restore.sh` (with `DATABASE_URL` set; the role needs `CREATEDB` for the verify)
Restore a dump	`sudo -u postgres pg_restore --clean --if-exists --no-owner --role=universe -d universe <file>.dump`
Rollback code	as `universe`: `git checkout <old commit>` → `npm ci` (root, client `--include=dev`, server) → `npm run build --prefix client` → `pm2 reload ecosystem.config.cjs`

Appendix B — Sign-off record

Item	Value
Date / time of switch	
Performed by	
Old commit (`modelling_tool`)	
New commit (`Saudi_re` `main`)	
Migrations applied (files)	
Reference counts (F1)	countries …… currencies …… brokers …… reinsurers …… cedants ……
Business counts before / after (S6)	contracts …… / …… quotes …… / …… claims …… / …… users …… / ……

Item	Value
Smoke test (R8)	PASSED ☐
Checklist §7.7 complete	☐
Backup taken after switch (P3)	file:
Token / deploy-key expiry	